



SideChannelKernelPreventor Driver Architecture Specification

Windows 11 x64 Ring 0 Kernel-Mode Driver (WDM/KMDF)

Version 1.1

Revision Date: May 2026

Classification: Internal Technical Reference

Test & Development Build

This driver requires test-signing mode (`bcdedit /set testsigning on`).

Deploy only in isolated test virtual machines. Not for production use.

Table of Contents

1 Project Overview	
2 Architecture Overview	
2.1 Zero-Trust Kernel Assumption	
2.2 Module Interaction Model	
3 Module 1 — Side-Channel Attack Detection	
3.1 PMU Configuration via MSRs	
3.2 Cache-Miss Rate Monitoring	
3.3 RDTSC Rate Anomaly Detection	
3.4 PMI Handler Constraints	
4 Module 2 — Hardware Keylogger & Peripheral Detection	
4.1 PCIe ECAM Walk	
4.2 VT-d IOMMU Verification	
4.3 Keyboard Filter Driver Detection	
5 Module 3 — Kernel Integrity & Hook Detection	
5.1 IDT Integrity Verification	
5.2 Dispatch-Routine Hook Detection	
5.3 SHA-256 .text Section Verification	
6 Module 4 — Cache Mitigation Engine	
6.1 CPU Capability Probing	
6.2 Sensitive Page Tagging	
6.3 VERW / L1D Flush Strategy	
6.4 Cache Allocation Technology (CAT)	
6.5 SMT / HT Isolation	
7 Module 5 — Secure Ring 0 ↔ Ring 3 Communication	
7.1 HMAC-Authenticated Shared Memory	
7.2 MSR-Based Covert Signaling (Fallback)	
7.3 TPM 2.0 Key Derivation	
8 Cross-Module Event Flow & Performance	
8.1 Event Dispatch Table	
8.2 IRQL Constraints Summary	
8.3 Performance Targets	
9 Literature Foundation	

NEW

9.1 Textbooks and Monographs
9.2 Selected Academic Papers
9.3 Standards and Technical Reports
9.4 Courses and Lecture Series

10 Literature Map **NEW**

A Appendix A — Threat Model Maintenance **NEW**

B Appendix B — Prior Art Analysis **NEW**

1 Project Overview

SideChannelKernelPreventor is a Windows 11 x64 Ring 0 kernel-mode driver (WDM/KMDF) designed to detect and actively mitigate side-channel attacks, compromised kernel components, and hardware-level keyloggers. It is structured as five synergistic modules that share a central integrity-verified state structure protected by spin locks at **DISPATCH_LEVEL**.

The driver's threat model covers speculative-execution attacks (Spectre [P1], Meltdown [P2], Foreshadow [P3]), microarchitectural data-sampling (MDS) attacks such as RIDL [P4] and ZombieLoad [P5], classic cache-timing attacks [P7] [P8] [P9], and physical-layer threats including DMA-capable rogue peripherals and hardware keyloggers.

Module	Domain	Key Threat Addressed
M1	PMU / cache anomaly detection	Spectre [P1], Flush+Reload [P7], Flush+Flush [P8]
M2	PCIe / IOMMU verification	DMA attacks, rogue peripherals [S13]
M3	Kernel integrity & hook detection	Meltdown [P2], rootkit hooks [S12]
M4	Cache mitigation engine	MDS [P4] [P5], L1TF [P3], SMT cross-contamination [P11]
M5	Secure Ring 0 ↔ Ring 3 channel	Tamper-proof alert delivery [S3] [S7]

2 Architecture Overview

2.1 Zero-Trust Kernel Assumption

The driver operates under a *zero-trust kernel assumption*: it cannot rely on standard kernel APIs for critical integrity operations because the kernel itself may be partially compromised. Consequences of this assumption are:

- MSR reads and writes use inline MASM intrinsics — never a wrapper that could be interposed.
- PCIe configuration space is accessed via ECAM physical addresses, bypassing the OS device manager.
- HMAC authentication of all Ring 3 notifications prevents spoofed alerts even if DeviceIoControl dispatch is hooked.
- SHA-256 of kernel `.text` sections is compared against TPM-sealed baselines [S3].

2.2 Module Interaction Model

All modules share `g_DriverState`, a central `DRIVER_STATE` structure. Access requires acquiring `g_DriverState.StateLock` at **DISPATCH_LEVEL**. Cross-module events are dispatched via `DispatchCrossModuleEvent()` in `shared_state.c`.

```
// shared_state.h – Central state (abbreviated)
typedef struct _DRIVER_STATE {
    KSPIN_LOCK      StateLock;           // DISPATCH_LEVEL guard
    ULONG           ActiveMitigations;   // Bitmask of enabled mitigations
    LONGLONG        LastIntegrityCheck;  // 100-ns ticks
    BOOLEAN          FailSafeMode;       // Set on .text hash mismatch
    CPU_MITIGATION  g_CpuMit[MAXIMUM_PROCESSORS]; // Per-core strategy
} DRIVER_STATE;

extern DRIVER_STATE g_DriverState;

VOID DispatchCrossModuleEvent(
    _In_ ULONG      SourceModule,  // MODULE_ID_M1 ... MODULE_ID_M5
    _In_ EVENT_TYPE Event,
    _In_ ULONG_PTR  Parameter
);
```

Warning: StateLock must be acquired before any read or write of g_DriverState fields. Failure to do so causes a DRIVER_IRQL_NOT_LESS_OR_EQUAL bugcheck.

3 Module 1 — Side-Channel Attack Detection (PMU)

Module 1 instruments the CPU's Performance Monitoring Unit (PMU) to detect cache-side-channel and timing-based attack patterns in real time. It runs primarily at **DISPATCH_LEVEL** for configuration and at **HIGH_LEVEL** inside the PMI handler. [S1] [S2] [L5]

3.1 PMU Configuration via MSRs

Four PMU counters are configured at driver load time. Event selection uses `IA32_PERFEVTSEL0-3` (MSR 0x186–0x189); counter values are read from `IA32_PMC0-3` (MSR 0x0C1–0x0C4). [S1]

```
// pmu_detection.c
// MSR addresses – Intel SDM Vol.4 §18.2.1 [S1]
#define MSR_PERFEVTSEL_BASE 0x186UL
#define MSR_PMC_BASE        0x0C1UL

// IA32_PERFEVTSELx bit layout
// [7:0] Event Select [15:8] Unit Mask (UMASK)
// [16] USR [17] OS [22] EN [20] INT
#define PMU_EVT_LLC_MISS    0x412E00UL // LLC references, OS+USR, EN, INT
#define PMU_EVT_L1D_MISS    0x4F2408UL // L1D read misses, OS+USR, EN, INT
#define PMU_EVT_BR_MISP     0x4FC400UL // Branch mispredictions, OS+USR, EN, INT
#define PMU_EVT_RDTSC_RETIRE 0x410032UL // RDTSC retired instructions, USR only, EN, INT

static VOID
ConfigurePmuCounters(VOID)
{
    // Counter 0 – LLC miss rate [P7][P8]
    __writemsr(MSR_PERFEVTSEL_BASE + 0, PMU_EVT_LLC_MISS);
    __writemsr(MSR_PMC_BASE + 0, 0ULL);

    // Counter 1 – L1D miss rate [P9][P10]
    __writemsr(MSR_PERFEVTSEL_BASE + 1, PMU_EVT_L1D_MISS);
    __writemsr(MSR_PMC_BASE + 1, 0ULL);

    // Counter 2 – Branch misprediction (Spectre indicator) [P1]
    __writemsr(MSR_PERFEVTSEL_BASE + 2, PMU_EVT_BR_MISP);
    __writemsr(MSR_PMC_BASE + 2, 0ULL);

    // Counter 3 – RDTSC retirement rate (timing-channel probe) [P10][P12]
    __writemsr(MSR_PERFEVTSEL_BASE + 3, PMU_EVT_RDTSC_RETIRE);
    __writemsr(MSR_PMC_BASE + 3, 0ULL);
}
```

3.2 Cache-Miss Rate Monitoring

Flush+Reload [P7] and Flush+Flush [P8] attacks produce a characteristic LLC miss rate spike. M1 samples PMC0 and PMC1 inside the PMI handler and raises a cross-module event when the ratio exceeds a calibrated threshold.

```

// pmu_detection.c
// Thresholds derived from measurement on target hardware per [P7] §5, [P10]
#define LLC_MISS_THRESHOLD_PER_MS 180000ULL
#define L1D_MISS_THRESHOLD_PER_MS 1200000ULL
#define RDTSC_RATE_THRESHOLD_PER_MS 8000ULL

static VOID
AnalyzePmuCounters(
    _In_ ULONGLONG LlcMisses,
    _In_ ULONGLONG L1dMisses,
    _In_ ULONGLONG RdtscRetired
)
{
    if (LlcMisses > LLC_MISS_THRESHOLD_PER_MS ||
        L1dMisses > L1D_MISS_THRESHOLD_PER_MS)
    {
        // Probable Flush+Reload or Flush+Flush activity [P7][P8]
        DispatchCrossModuleEvent(
            MODULE_ID_M1,
            EVENT_HIGH_CACHE_MISS_RATE,
            (ULONG_PTR)LlcMisses);
    }

    if (RdtscRetired > RDTSC_RATE_THRESHOLD_PER_MS)
    {
        // Probable timing-channel probe; alert Ring 3 [P10][P12]
        DispatchCrossModuleEvent(
            MODULE_ID_M1,
            EVENT_RDTSC_ANOMALY,
            (ULONG_PTR)RdtscRetired);
    }
}

```

3.3 PMI Handler Constraints

Warning: The PMI handler executes at HIGH_LEVEL IRQL. No paged memory, no kernel API calls, no spinlock acquisitions. Read only non-paged globals and MSRs.

```
// pmu_detection.c - PMI interrupt handler
// IRQL: HIGH_LEVEL - zero paged memory, no KAPI, no locks [L6][S1]
BOOLEAN
PmiIsr(
    _In_ PKINTERRUPT Interrupt,
    _In_ PVOID ServiceContext
)
{
    UNREFERENCED_PARAMETER(Interrupt);
    UNREFERENCED_PARAMETER(ServiceContext);

    // Read counters at HIGH_LEVEL using MSR intrinsics (not hookcable) [S1]
    ULONGLONG llc = __readmsr(MSR_PMC_BASE + 0);
    ULONGLONG l1d = __readmsr(MSR_PMC_BASE + 1);
    ULONGLONG rdtsc = __readmsr(MSR_PMC_BASE + 3);

    // Reset counters
    __writemsr(MSR_PMC_BASE + 0, 0ULL);
    __writemsr(MSR_PMC_BASE + 1, 0ULL);
    __writemsr(MSR_PMC_BASE + 3, 0ULL);

    // Record in non-paged ring buffer (no allocation, no lock)
    ULONG idx = (ULONG)InterlockedIncrement((LONG*)&g_PmiRing.WriteIdx)
        & (PMI_RING_SIZE - 1);
    g_PmiRing.Samples[idx].LlcMisses = llc;
    g_PmiRing.Samples[idx].L1dMisses = l1d;
    g_PmiRing.Samples[idx].RdtscRetired = rdtsc;

    // Queue a DPC at DISPATCH_LEVEL for analysis (safe to call locks there)
    IoRequestDpc(g_DeviceObject, NULL, NULL);

    return TRUE;
}
```

4 Module 2 — Hardware Keylogger & Peripheral Detection

Module 2 detects unauthorized hardware by walking the PCIe topology directly via ECAM (bypassing the OS device stack), verifying Intel VT-d IOMMU configuration, and comparing hardware-reported keyboard devices against the Windows driver stack. [S13] [S14]

4.1 PCIe ECAM Walk

ECAM maps PCIe configuration space at a physical base address in 256 MB blocks (one 4 KB page per device function). The driver reads `MmMapIoSpace` at **PASSIVE_LEVEL** to obtain a kernel virtual address. [S13]

```

// hw_keylogger_detect.c
// PCIe Enhanced Configuration Access Mechanism – PCIe Base Spec Rev 5.0 §7.2 [S13]
// ECAM formula: PhysBase | (Bus << 20) | (Dev << 15) | (Func << 12)
#define ECAM_BASE_PHYS 0xE0000000ULL // Read from MCFG ACPI table at runtime
#define ECAM_CFG_SIZE 0x10000000ULL // 256 MB covers all 256 buses

typedef struct _PCIE_CFG_SPACE {
    USHORT VendorId; // 0x00
    USHORT DeviceId; // 0x02
    USHORT Command; // 0x04
    USHORT Status; // 0x06
    UCHAR RevisionId; // 0x08
    UCHAR ClassCode[3]; // 0x09 [2]=Base [1]=Sub [0]=ProgIf
    // ... (standard header)
} PCIE_CFG_SPACE, *PPCIE_CFG_SPACE;

// ClassCode[2] == 0x09 → Input device controller
// ClassCode[1] == 0x00 → Keyboard controller
#define PCI_CLASS_INPUT_KEYBOARD 0x09
#define PCI_SUBCLASS_KEYBOARD 0x00

static NTSTATUS
WalkPcieBusTree(
    _In_ PVOID EcamVirtBase,
    _Out_ PULONG UnauthorizedDeviceCount
)
{
    *UnauthorizedDeviceCount = 0;

    for (ULONG bus = 0; bus <= 255; bus++) {
        for (ULONG dev = 0; dev < 32; dev++) {
            for (ULONG func = 0; func < 8; func++) {

                ULONG_PTR offset = ((ULONG_PTR)bus << 20) |
                                    ((ULONG_PTR)dev << 15) |
                                    ((ULONG_PTR)func << 12);
                volatile PPCIE_CFG_SPACE cfg =
                    (PPCIE_CFG_SPACE)((UCHAR*)EcamVirtBase + offset);

                if (cfg->VendorId == 0xFFFF)
                    continue; // No device

                if (cfg->ClassCode[2] == PCI_CLASS_INPUT_KEYBOARD &&
                    cfg->ClassCode[1] == PCI_SUBCLASS_KEYBOARD)
                {
                    if (!IsDeviceAuthorized(cfg->VendorId, cfg->DeviceId))
                    {
                        (*UnauthorizedDeviceCount)++;
                        LogAlert(ALERT_LEVEL_CRITICAL,
                            "Unauthorized keyboard device: VID=%04X DID=%04X "
                            "Bus=%u Dev=%u Func=%u",
                            cfg->VendorId, cfg->DeviceId, bus, dev, func);
                    }
                }
            }
        }
    }
}

```

```
    return STATUS_SUCCESS;
}
```

4.2 VT-d IOMMU Verification

The driver reads the VT-d Capability Register to verify IOMMU is enabled and that no bypass conditions exist. If a DMA violation is detected, M4 is notified to lock the IOMMU context entry. [S1]

```
// hw_keylogger_detect.c – IOMMU capability check
// Intel VT-d Architecture Specification §11.4.2
#define VTDUNIT_PHYS_BASE 0xFED90000ULL // Typical default; read DMAR ACPI table

#define VTD_REG_CAP      0x08 // Capability Register offset
#define VTD_REG_ECAP     0x10 // Extended Capability Register offset
#define VTD_CAP_DRD      (1ULL << 55) // DMA Read Draining supported
#define VTD_CAP_ESIRTPS  (1ULL << 29) // Enhanced Set Interrupt Remap TPS

static BOOLEAN
VerifyIommuEnabled(
    _In_ PVOID VtdVirtBase
)
{
    volatile ULONGLONG cap = *(volatile ULONGLONG*)((UCHAR*)VtdVirtBase + VTD_REG_CAP);
    volatile ULONGLONG ecap = *(volatile ULONGLONG*)((UCHAR*)VtdVirtBase + VTD_REG_ECAP);

    BOOLEAN dmaRemapActive = (ecap & 0x1) != 0; // EIM: Extended Interrupt Mode
    BOOLEAN drainSupported = (cap & VTD_CAP_DRD) != 0;

    if (!dmaRemapActive) {
        LogAlert(ALERT_LEVEL_HIGH, "IOMMU DMA remapping inactive – DMA attacks possible");
        return FALSE;
    }

    UNREFERENCED_PARAMETER(drainSupported);
    return TRUE;
}
```

4.3 Keyboard Filter Driver Detection

The OS driver stack for each keyboard device is enumerated via `IoGetDeviceProperty` at **PASSIVE_LEVEL**. Each filter driver's image path is compared against an allowlist of Microsoft-signed components. Unauthorized entries trigger an alert and a cross-module event. [S12]

5 Module 3 — Kernel Integrity & Hook Detection

Module 3 detects runtime modifications to the kernel's control flow including IDT hooking, inline detours on keyboard dispatch routines, and alterations to kernel `.text` sections. All comparisons use **constant-time algorithms** to prevent timing leakage per Kocher [P12]. [S5] [S6] [S12]

5.1 IDT Integrity Verification

```
// kernel_integrity.c
// IDT Gate layout – Intel SDM Vol.3A §6.11 [S1]
#pragma pack(push, 1)
typedef struct _IDT_GATE {
    USHORT OffsetLow;    // [15:0]
    USHORT Selector;     // Segment selector
    UCHAR Ist;          // Interrupt Stack Table index (bits [2:0])
    UCHAR Type;         // Gate type (0x8E = 64-bit interrupt gate)
    USHORT OffsetMid;    // [31:16]
    ULONG OffsetHigh;    // [63:32]
    ULONG Reserved;
} IDT_GATE, *PIDT_GATE;
#pragma pack(pop)

// IRQ 1 = keyboard (PIC line 1 mapped to vector 0x31 on typical Windows)
#define KEYBOARD_IRQ_VECTOR 0x31

static NTSTATUS
VerifyIdtIntegrity(VOID)
{
    IDTR idtr;
    __sidt(&idtr);    // Read IDTR directly – no kernel wrapper

    PIDT_GATE idt = (PIDT_GATE)(ULONG_PTR)idtr.Base;
    PIDT_GATE gate = &idt[KEYBOARD_IRQ_VECTOR];

    ULONG_PTR handler = ((ULONG_PTR)gate->OffsetHigh << 32) |
        ((ULONG_PTR)gate->OffsetMid << 16) |
        (ULONG_PTR)gate->OffsetLow;

    // Handler must reside in ntoskrnl.exe image range
    if (handler < g_KernelBase || handler >= g_KernelBase + g_KernelSize) {
        LogAlert(ALERT_LEVEL_CRITICAL,
            "IDT vector 0x%02X handler 0x%11X outside kernel range",
            KEYBOARD_IRQ_VECTOR, (ULONGLONG)handler);

        DispatchCrossModuleEvent(MODULE_ID_M3, EVENT_IDT_HOOK_DETECTED, handler);
        return STATUS_UNSUCCESSFUL;
    }
    return STATUS_SUCCESS;
}
```

5.2 Dispatch-Routine Hook Detection

Inline hooks (detours) on `Kbclass` and `Kbdhid` `IRP_MJ_READ` dispatch routines are detected by comparing the first 16 bytes of the function against an in-memory copy taken at driver load time. [P2] [S12]

```

// kernel_integrity.c
#define DETOUR_SNAPSHOT_BYTES 16

// Baseline snapshot taken at DriverEntry – before any potential hooking
static UCHAR g_KbclassReadBaseline[DETOUR_SNAPSHOT_BYTES];
static ULONG_PTR g_KbclassReadAddr;

static NTSTATUS
SnapshotDispatchBaseline(
    _In_ PDRIVER_OBJECT KbclassDriver
)
{
    g_KbclassReadAddr =
        (ULONG_PTR)KbclassDriver->MajorFunction[IRP_MJ_READ];
    RtlCopyMemory(g_KbclassReadBaseline,
        (PVOID)g_KbclassReadAddr,
        DETOUR_SNAPSHOT_BYTES);
    return STATUS_SUCCESS;
}

static BOOLEAN
CheckForDetour(VOID)
{
    UCHAR current[DETOUR_SNAPSHOT_BYTES];
    RtlCopyMemory(current, (PVOID)g_KbclassReadAddr, DETOUR_SNAPSHOT_BYTES);

    // Constant-time comparison prevents timing oracle [P12]
    UCHAR diff = 0;
    for (ULONG i = 0; i < DETOUR_SNAPSHOT_BYTES; i++)
        diff |= (current[i] ^ g_KbclassReadBaseline[i]);

    return (diff != 0);
}

```

5.3 SHA-256 .text Section Verification

The SHA-256 digest of `ntoskrnl.exe`'s `.text` section is computed using BCrypt at **PASSIVE_LEVEL** and compared against a TPM-sealed baseline [S3]. Verification is staggered across DPC callbacks to limit performance impact.

```

// kernel_integrity.c - SHA-256 .text verification [S3][P2]
// Runs at PASSIVE_LEVEL via a work item (DPC queues the work item)
static VOID
VerifyKernelTextHash(
    _In_ PDEVICE_OBJECT DeviceObject,
    _In_ PVOID Context
)
{
    UNREFERENCED_PARAMETER(DeviceObject);
    UNREFERENCED_PARAMETER(Context);

    BCryptAlgHandle hAlg = NULL;
    BCryptHashHandle hHash = NULL;
    UCHAR digest[32] = {0};
    ULONG cbHash = sizeof(digest);
    ULONG cbObject = 0, cbData = 0;

    // BCrypt requires PASSIVE_LEVEL - do not call from DPC directly [L6]
    BCryptOpenAlgorithmProvider(&hAlg, BCryptSHA256_ALGORITHM, NULL, 0);
    BCryptGetProperty(hAlg, BCryptObjectLength,
        (PUCHAR)&cbObject, sizeof(ULONG), &cbData, 0);

    PVOID pHashObj = ExAllocatePoolWithTag(NonPagedPoolNx, cbObject, 'HASH');
    BCryptCreateHash(hAlg, &hHash, pHashObj, cbObject, NULL, 0, 0);
    BCryptHashData(hHash, (PUCHAR)g_KernelTextBase, g_KernelTextSize, 0);
    BCryptFinishHash(hHash, digest, cbHash, 0);

    // Constant-time comparison against TPM-sealed baseline [P12][S3]
    UCHAR diff = 0;
    for (ULONG i = 0; i < 32; i++)
        diff |= (digest[i] ^ g_KernelTextBaseline[i]);

    if (diff != 0) {
        LogAlert(ALERT_LEVEL_CRITICAL, ".text hash mismatch - kernel may be compromised");
        DispatchCrossModuleEvent(MODULE_ID_M3, EVENT_TEXT_HASH_MISMATCH, 0);
    }

    BCryptDestroyHash(hHash);
    BCryptCloseAlgorithmProvider(hAlg, 0);
    ExFreePoolWithTag(pHashObj, 'HASH');
}

```

6 Module 4 — Cache Mitigation Engine

Module 4 is the core mitigation engine. It does not attempt to identify cryptographic code — it protects *any* sensitive memory based on a tagging discipline. It uses `VERW / IA32_FLUSH_CMD` flushes, CAT cache partitioning, IBRS/STIBP speculation controls, and SMT isolation to neutralize MDS [\[P4\]](#) [\[P5\]](#), L1TF [\[P3\]](#) [\[S11\]](#), and Spectre [\[P1\]](#) [\[S9\]](#) attack vectors.

6.1 CPU Capability Probing

At driver load the CPU's vulnerability profile is read from `IA32_ARCH_CAPABILITIES` (MSR 0x10A) [\[S1\]](#) [\[S10\]](#) [\[S11\]](#). This determines the cheapest safe mitigation strategy for the current processor.

```

// cache_mitigation.c
// IA32_ARCH_CAPABILITIES bits – Intel SDM Vol.4 §2.1, Table 2-2 [S1]
// Intel SA-00233 §3.1 [S10], Intel SA-00161 §2.1 [S11]
#define MSR_IA32_ARCH_CAPABILITIES 0x10AUL
#define ARCH_CAP_RDCL_NO (1ULL << 0) // Not vulnerable to Meltdown/RDCL
#define ARCH_CAP_IBRS_ALL (1ULL << 1) // Enhanced IBRS supported
#define ARCH_CAP_RSBA (1ULL << 2) // RSB alternate branch prediction
#define ARCH_CAP_SKIP_L1DFL (1ULL << 3) // L1D flush not required for VMX
#define ARCH_CAP_MDS_NO (1ULL << 5) // Not vulnerable to MDS [S10]
#define ARCH_CAP_TAA_NO (1ULL << 8) // Not vulnerable to TSX async abort

typedef enum _MITIGATION_STRATEGY {
    MIT_NONE = 0, // Not vulnerable
    MIT_VERW_ONLY = 1, // MDS only: VERW on domain exit [P4][P5]
    MIT_L1D_FLUSH = 2, // L1TF: IA32_FLUSH_CMD on VM entry [P3]
    MIT_VERW_AND_L1D = 3, // Both MDS and L1TF present
    MIT_FULL_SPECTRUM = 4, // Hash mismatch fail-safe: everything
} MITIGATION_STRATEGY;

static VOID
ProbeCpuCapabilities(
    _Out_ PMITIGATION_STRATEGY Strategy
)
{
    ULONGLONG archCap = 0;

    // CPUID.7.0:EDX[29] = IA32_ARCH_CAPABILITIES availability check
    int cpuidResult[4];
    __cpuidex(cpuidResult, 7, 0);
    BOOLEAN hasArchCap = ((cpuidResult[3] >> 29) & 1) != 0;

    if (hasArchCap) {
        archCap = __readmsr(MSR_IA32_ARCH_CAPABILITIES);

        if (archCap & ARCH_CAP_MDS_NO)
            g_CpuMit[KeGetCurrentProcessorNumber()].Features |= CPU_FEAT_MDS_NO;

        if (archCap & ARCH_CAP_RDCL_NO)
            g_CpuMit[KeGetCurrentProcessorNumber()].Features |= CPU_FEAT_L1TF_NO;
    }

    BOOLEAN needVerw = !(archCap & ARCH_CAP_MDS_NO);
    BOOLEAN needL1d = !(archCap & ARCH_CAP_RDCL_NO) &&
        !(archCap & ARCH_CAP_SKIP_L1DFL);

    if (!needVerw && !needL1d) *Strategy = MIT_NONE;
    else if (needVerw && !needL1d) *Strategy = MIT_VERW_ONLY;
    else if (!needVerw && needL1d) *Strategy = MIT_L1D_FLUSH;
    else *Strategy = MIT_VERW_AND_L1D;
}

```

6.2 Sensitive Page Tagging

Callers tag pages containing keystroke buffers, key material, or crypto I/O using `ScpdTagSensitivePage()`. Tagged pages are tracked in a non-paged bitmap and are excluded from user-mode page tables when not strictly required (KPTI-style). [P2] [S12]

```

// cache_mitigation.c - Sensitive page API
// Based on KPTI principles from Meltdown mitigation [P2][S12]
#define MAX_SENSITIVE_PAGES 4096
static ULONG_PTR g_SensitivePages[MAX_SENSITIVE_PAGES];
static ULONG g_SensitivePageCount = 0;
static KSPIN_LOCK g_SensitivePagesLock;

NTSTATUS
ScpdTagSensitivePage(
    _In_ PVOID VirtualAddress,
    _In_ BOOLEAN Sensitive
)
{
    KIRQL oldIrql;
    KeAcquireSpinLock(&g_SensitivePagesLock, &oldIrql);

    ULONG_PTR pfn = MmGetPhysicalAddress(VirtualAddress).QuadPart >> PAGE_SHIFT;

    if (Sensitive) {
        if (g_SensitivePageCount < MAX_SENSITIVE_PAGES) {
            g_SensitivePages[g_SensitivePageCount++] = pfn;
        }
    } else {
        for (ULONG i = 0; i < g_SensitivePageCount; i++) {
            if (g_SensitivePages[i] == pfn) {
                g_SensitivePages[i] =
                    g_SensitivePages[--g_SensitivePageCount];
                break;
            }
        }
    }

    KeReleaseSpinLock(&g_SensitivePagesLock, oldIrql);
    return STATUS_SUCCESS;
}

```

6.3 VERW / L1D Flush Strategy

PerformVerwFlush() is implemented in MASM (asm/verw_flush.asm) because MSVC has no intrinsic for VERW . It is the only location in the codebase where the VERW instruction appears. [P4] [P5] [S10]

```
// asm/verw_flush.asm
; PerformVerwFlush – MFENCE + VERW to clear MDS buffers
; Called on every context-switch boundary touching a tagged page
; Intel SA-00233 §4: VERW with any writable data segment [S10]
; VERW 0x2B = user data segment selector on standard Windows x64

.CODE

PerformVerwFlush PROC
    mfence                ; Serialize prior stores [P4]
    sub    rsp, 2
    mov    WORD PTR [rsp], 02Bh ; User data segment selector
    verw   WORD PTR [rsp]      ; Clear CPU line/load buffers [P5][S10]
    add    rsp, 2
    mfence
    ret
PerformVerwFlush ENDP

END
```

```
// cache_mitigation.c – L1D flush for L1TF [P3][S11]
// IA32_FLUSH_CMD MSR 0x10B – Intel SDM Vol.4 §2.1 [S1]
#define MSR_IA32_FLUSH_CMD    0x10BUL
#define FLUSH_CMD_L1D_FLUSH  0x1ULL

static FORCEINLINE VOID
PerformL1dFlush(VOID)
{
    __writemsr(MSR_IA32_FLUSH_CMD, FLUSH_CMD_L1D_FLUSH);
}

// Entry point called by context-switch callback
VOID
ScpdApplyMitigationOnContextSwitch(
    _In_ BOOLEAN TouchesSensitivePages,
    _In_ MITIGATION_STRATEGY Strategy
)
{
    if (!TouchesSensitivePages && Strategy < MIT_FULL_SPECTRUM)
        return;

    switch (Strategy) {
        case MIT_VERW_ONLY:      PerformVerwFlush();           break;
        case MIT_L1D_FLUSH:      PerformL1dFlush();            break;
        case MIT_VERW_AND_L1D:   PerformVerwFlush(); PerformL1dFlush(); break;
        case MIT_FULL_SPECTRUM:  PerformVerwFlush(); PerformL1dFlush();
                                ScpdApplySpeculationControls(TRUE); break;
        default:                  break;
    }
}
```

6.4 Cache Allocation Technology (CAT)

On processors supporting Intel CAT, Module 4 partitions the L3 cache to create an isolated LLC region for sensitive workloads. This prevents LLC-based Flush+Reload cross-contamination. [P7] [S1]

```

// cache_mitigation.c – CAT / RDT setup
// Intel SDM Vol.3B §17.18 [S1]
#define MSR_PQR_ASSOC    0xC8FUL // Class Of Service assignment
#define MSR_L3_CBMBASE   0xC90UL // L3 capacity bit mask base (CLOS 0)
#define MSR_L3_CBMBASE_1 0xC91UL // CLOS 1 – sensitive workloads

// Bit mask: lower half of LLC ways → normal, upper half → sensitive
// Example: 0x003FF = ways 0-9 (normal), 0xFFC00 = ways 10-19 (sensitive)
#define CAT_CBM_NORMAL    0x003FFUL
#define CAT_CBM_SENSITIVE 0xFFC00UL
#define CLOS_NORMAL       0
#define CLOS_SENSITIVE    1

static NTSTATUS
ConfigureCatPartition(VOID)
{
    // Verify CAT support: CPUID.10H.1:EAX[4:0] = CBM length
    int cpuidResult[4];
    __cpuidex(cpuidResult, 0x10, 1);
    ULONG cbmLen = (cpuidResult[0] & 0x1F) + 1;
    if (cbmLen < 10)
        return STATUS_NOT_SUPPORTED;

    __writemsr(MSR_L3_CBMBASE, CAT_CBM_NORMAL);
    __writemsr(MSR_L3_CBMBASE_1, CAT_CBM_SENSITIVE);

    // Assign current core to sensitive CLOS when processing tagged pages
    __writemsr(MSR_PQR_ASSOC, (ULONGLONG)CLOS_SENSITIVE);

    return STATUS_SUCCESS;
}

```

6.5 SMT / HT Isolation

When a logical core processes tagged memory, its sibling hyper-thread is restricted and an L1D flush is issued before domain transitions. This prevents cross-HT cache timing leakage per Percival [P11] and the MDS threat model [P4] [P5].

```
// cache_mitigation.c - SMT sibling isolation [P11][P4]
// IA32_SPEC_CTRL MSR 0x48 - Intel SDM Vol.4 §2.1 [S1][S9]
#define MSR_IA32_SPEC_CTRL 0x48UL
#define SPEC_CTRL_IBRS      (1ULL << 0) // Indirect Branch Restricted Spec.
#define SPEC_CTRL_STIBP     (1ULL << 1) // Single Thread IB Prediction [P1]

static VOID
ScpdApplySpeculationControls(
    _In_ BOOLEAN Enable
)
{
    ULONGLONG ctrl = __readmsr(MSR_IA32_SPEC_CTRL);

    if (Enable)
        ctrl |= (SPEC_CTRL_IBRS | SPEC_CTRL_STIBP);
    else
        ctrl &= ~(SPEC_CTRL_IBRS | SPEC_CTRL_STIBP);

    __writemsr(MSR_IA32_SPEC_CTRL, ctrl);
}
```

7 Module 5 — Secure Ring 0 ↔ Ring 3 Communication

Module 5 provides the Ring 3 monitor process (`SideChannelMonitor.exe`) with a tamper-proof notification channel. The primary channel uses HMAC-authenticated shared memory; the fallback uses MSR-based covert signaling for critical alerts when `DeviceIoControl` dispatch may be compromised. [\[S3\]](#) [\[S7\]](#) [\[S4\]](#)

7.1 HMAC-Authenticated Shared Memory

```
// scpd_shared.h – shared structures (kernel + user)
// IOCTL codes
#define IOCTL_SCPD_MAP_SHARED_MEM CTL_CODE(FILE_DEVICE_UNKNOWN, 0x800,
METHOD_BUFFERED, FILE_READ_ACCESS)
#define IOCTL_SCPD_GET_HMAC_KEY CTL_CODE(FILE_DEVICE_UNKNOWN, 0x801,
METHOD_BUFFERED, FILE_READ_ACCESS)

#define SHARED_RING_SIZE 256 // Power of 2

typedef struct _ALERT_ENTRY {
    LARGE_INTEGER Timestamp; // 100-ns ticks
    ULONG AlertLevel; // 0=info 1=warning 2=critical
    ULONG EventType; // EVENT_TYPE enum
    CHAR Details[120]; // UTF-8 message
    UCHAR Hmac[32]; // HMAC-SHA256(key, entry sans Hmac) [S7]
} ALERT_ENTRY, *PALERT_ENTRY;

typedef struct _SHARED_MEM_REGION {
    volatile ULONG WriteIndex; // Producer writes; reader polls
    volatile ULONG ReadIndex; // Reader advances
    ALERT_ENTRY Ring[SHARED_RING_SIZE];
} SHARED_MEM_REGION, *PSHARED_MEM_REGION;
```

```

// secure_comms.c – HMAC key derivation [S3][S7][S4]
// Key is sealed by TPM 2.0 PCR policy; retrieved at driver load
// NIST SP 800-57 Pt.1 §5.3.4: 256-bit HMAC key for SHA-256 [S7]
static UCHAR g_HmacKey[32] = {0};

static NTSTATUS
DeriveHmacKeyFromTpm(VOID)
{
    // TPM2_Create with policyPCR binding (PCR 7 = Secure Boot state) [S3]
    // Implementation uses TBS (TPM Base Services) from Ring 3 at setup time
    // and seals the key – driver retrieves via IOCTL_SCPD_GET_HMAC_KEY
    return TpmUnsealKey(g_HmacKey, sizeof(g_HmacKey));
}

VOID
WriteAlertToRing(
    _In_ ULONG Level,
    _In_ ULONG EventType,
    _In_ PCSTR Details
)
{
    ULONG idx = InterlockedIncrement((LONG*)&g_SharedMem->WriteIndex)
                & (SHARED_RING_SIZE - 1);

    PALERT_ENTRY entry = &g_SharedMem->Ring[idx];
    entry->Timestamp = KeQueryPerformanceCounter(NULL);
    entry->AlertLevel = Level;
    entry->EventType = EventType;
    RtlStringCbCopyA(entry->Details, sizeof(entry->Details), Details);
    RtlZeroMemory(entry->Hmac, sizeof(entry->Hmac));

    // HMAC-SHA256 using BCrypt [S7]
    ComputeHmacSha256(g_HmacKey, sizeof(g_HmacKey),
                      (PUCHAR)entry, sizeof(ALERT_ENTRY),
                      entry->Hmac);
}

```

7.2 MSR-Based Covert Signaling (Fallback)

For ultra-critical alerts (e.g., `.text` hash mismatch), a covert signal is written to an unused MSR that the monitor polls. This channel is resistant to `DeviceIoControl` interception.

```
// secure_comms.c - MSR covert channel (fallback)
// Uses a vendor-reserved MSR range; value is HMAC-truncated alert code
#define MSR_COVERT_SIGNAL 0x400000UL // AMD: OSVW range; unused on Intel

VOID
SendMsrCovertSignal(
    _In_ ULONG AlertCode
)
{
    // Pack alert code + 32-bit HMAC truncation into 64-bit MSR value
    ULONGLONG payload = ((ULONGLONG)AlertCode << 32) |
        ComputeHmacTrunc32(g_HmacKey, AlertCode);
    __writemsr(MSR_COVERT_SIGNAL, payload);
}
```

7.3 TPM 2.0 Key Derivation

Session HMAC keys are derived and sealed by the TPM at first boot. PCR 7 binds the key to the current Secure Boot state; a change in Secure Boot policy causes unsealing to fail and triggers fail-safe mode. [S3] [S7]

8 Cross-Module Event Flow & Performance

8.1 Event Dispatch Table

Trigger Source	Event	Response Module	Action
M1	High LLC/L1D miss rate	M4	Increase VERW flush frequency [P7] [P8]
M1	RDTSR rate anomaly	M5	Alert Ring 3 monitor [P10] [P12]
M2	Unauthorized DMA device	M4	Lock IOMMU context + flush [S13]
M2	IOMMU disabled	M5	Critical alert; optional fail-safe [S11]
M3	IDT keyboard hook	M4	Full-spectrum flush + core isolation [P2]
M3	.text hash mismatch	M4+M5	Maximum mitigations + covert MSR alert [S3]
M3	Dispatch routine detour	M5	Critical alert [P1]
M4	Shared state corruption	M4	Apply maximum mitigations immediately

8.2 IRQ Constraints Summary

IRQ	Allowable Operations	Prohibited
HIGH_LEVEL	MSR reads/writes, non-paged ring buffer writes, IoRequestDpc	Any kernel API, spin locks, paged memory
DISPATCH_LEVEL	Spin lock acquire/release, non-paged pool, DPC body	Paged memory, blocking calls, BCrypt
PASSIVE_LEVEL	Full kernel API, BCrypt, MmMapIoSpace, IoGetDeviceProperty	—

8.3 Performance Targets

Scenario	Target Overhead	Dominating Cost
No active attack	< 0.1 %	PMU polling DPC
MDS mitigation only (VERW)	~50 ns / context switch	VERW + MFENCE [P4]
MDS + L1TF (VERW + L1D flush)	~250 ns / context switch	IA32_FLUSH_CMD [P3] [S11]
SMT isolation enabled	5–15 % throughput reduction	Sibling-core stall + L1D flush [P11]
Full-spectrum (fail-safe)	~800 ns / context switch	IBRS + STIBP + both flushes

9 Literature Foundation

NEW

This section catalogs the bibliographic sources that underpin the driver's design decisions, threat model, and implementation choices. All implementation decisions should be traceable to at least one entry in this section. Sources are grouped by type following the taxonomy in *Principles of Working with Literature Documents*, §1.

9.1 Textbooks and Monographs

Operating Systems & Kernel Internals

#	Title	Author(s)	Ed.	Publisher / Year	ISBN-13
[1]	<i>Windows Internals, Part 1</i>	Russinovich, M.; Solomon, D.; Ionescu, A.; Yosifovich, P.	7th	Microsoft Press, 2017	978-0-7356-8418-7
[2]	<i>Windows Internals, Part 2</i>	Yosifovich, P.; Russinovich, M.; Solomon, D.; Ionescu, A.	7th	Microsoft Press, 2022	978-0-13-530285-1
[3]	<i>Modern Operating Systems</i>	Tanenbaum, A. S.; Bos, H.	4th	Pearson, 2014	978-0-13-359162-0
[4]	<i>Rootkits: Subverting the Windows Kernel</i>	Hoglund, G.; Butler, J.	1st	Addison-Wesley, 2005	978-0-321-29431-9
[5]	<i>Rootkits and Bootkits</i>	Matrosov, A.; Rodionov, E.; Bratus, S.	1st	No Starch Press, 2019	978-1-59327-716-1
[6]	<i>The Art of Memory Forensics</i>	Ligh, M. H.; Case, A.; Levy, J.; Walters, A.	1st	Wiley, 2014	978-1-118-82545-6
[7]	<i>Practical Malware Analysis</i>	Sikorski, M.; Honig, A.	1st	No Starch Press, 2012	978-1-59327-290-6

Computer Architecture & Microarchitecture

#	Title	Author(s)	Ed.	Publisher / Year	ISBN-13
[8]	<i>Computer Architecture: A Quantitative Approach</i>	Hennessy, J. L.; Patterson, D. A.	6th	Morgan Kaufmann, 2017	978-0-12-811905-1
[9]	<i>Computer Organization and Design (RISC-V ed.)</i>	Patterson, D. A.; Hennessy, J. L.	2nd	Morgan Kaufmann, 2020	978-0-12-820331-6
[10]	<i>Introduction to Hardware Security and Trust</i>	Tehraniipoor, M.; Wang, C. (Eds.)	1st	Springer, 2012	978-1-4419-8079-3

Cryptography & Applied Security

#	Title	Author(s)	Ed.	Publisher / Year	ISBN-13
[11]	<i>Applied Cryptography</i>	Schneier, B.	2nd	Wiley, 1996	978-0-471-11709-4
[12]	<i>Introduction to Modern Cryptography</i>	Katz, J.; Lindell, Y.	3rd	CRC Press, 2020	978-0-8153-8602-8
[13]	<i>A Practical Guide to TPM 2.0</i>	Arthur, W.; Challener, D.; Goldman, K.	1st	Apress, 2015	978-1-4302-6583-9

Secure Coding & Systems Security

#	Title	Author(s)	Ed.	Publisher / Year	ISBN-13
[14]	<i>Writing Secure Code</i>	Howard, M.; LeBlanc, D.	2nd	Microsoft Press, 2002	978-0-7356-1722-3
[15]	<i>Secure Coding in C and C++</i>	Seacord, R. C.	2nd	Addison-Wesley, 2013	978-0-321-82213-0
[16]	<i>Computer Security: Art and Science</i>	Bishop, M.	2nd	Pearson, 2018	978-0-321-71233-2
[17]	<i>Hacking: The Art of Exploitation</i>	Erickson, J.	2nd	No Starch Press, 2008	978-1-59327-144-2
[18]	<i>The Shellcoder's Handbook</i>	Anley, C.; Heasman, J.; Lindner, F.; Richarte, G.	2nd	Wiley, 2007	978-0-470-08023-8

9.2 Selected Academic Papers

Speculative Execution Side-Channel Attacks

[P1] Kocher, P. et al. (2019).
 "Spectre Attacks: Exploiting Speculative Execution."
 2019 IEEE Symposium on Security and Privacy (S&P), pp. 1-19.
 DOI: 10.1109/SP.2019.00002
 Relevance: M1 (PMU anomaly detection), M4 (LFENCE barriers, IBRS/STIBP via IA32_SPEC_CTRL)

[P2] Lipp, M. et al. (2018).
 "Meltdown: Reading Kernel Memory from User Space."
 27th USENIX Security Symposium, pp. 973-990.
 URL: <https://www.usenix.org/conference/usenixsecurity18/presentation/lipp>
 Relevance: M3 (kernel integrity verification), M4 (KPTI-style sensitive page isolation)

[P3] Van Bulck, J. et al. (2018).
"Foreshadow: Extracting the Keys to the Intel SGX Kingdom with Transient Out-of-Order Execution."
27th USENIX Security Symposium, pp. 991-1008.
URL: <https://www.usenix.org/conference/usenixsecurity18/presentation/bulck>
Relevance: M4 (L1TF mitigation – IA32_FLUSH_CMD MSR 0x10B)

Microarchitectural Data Sampling (MDS)

[P4] Van Schaik, S. et al. (2019).
"RIDL: Rogue In-flight Data Load."
2019 IEEE Symposium on Security and Privacy (S&P), pp. 88-105.
DOI: 10.1109/SP.2019.00087
Relevance: M4 (VERW buffer flush, MDS mitigation strategy selection)

[P5] Schwarz, M. et al. (2019).
"Zombieload: Cross-Privilege-Boundary Data Sampling."
ACM CCS 2019, pp. 753-768.
DOI: 10.1145/3319535.3354252
Relevance: M4 (MDS_NO capability flag interpretation, VERW effectiveness)

[P6] Canella, C. et al. (2019).
"A Systematic Evaluation of Transient Execution Attacks and Defenses."
28th USENIX Security Symposium, pp. 249-266.
URL: <https://www.usenix.org/conference/usenixsecurity19/presentation/canella>
Relevance: Comprehensive taxonomy; informs M4 mitigation selection logic

Cache-Timing Attacks

[P7] Yarom, Y.; Falkner, K. (2014).
"FLUSH+RELOAD: A High Resolution, Low Noise, L3 Cache Side-Channel Attack."
23rd USENIX Security Symposium, pp. 719-732.
URL: <https://www.usenix.org/conference/usenixsecurity14/technical-sessions/presentation/yarom>
Relevance: M1 (LLC miss rate detection threshold calibration)

[P8] Gruss, D.; Maurice, C.; Wagner, K.; Mangard, S. (2016).
"Flush+Flush: A Fast and Stealthy Cache Attack."
DIMVA 2016, LNCS 9721, pp. 279-299.
DOI: 10.1007/978-3-319-40667-1_14
Relevance: M1 (detection of low-noise, stealthy timing variants)

[P9] Osvik, D. A.; Shamir, A.; Tromer, E. (2006).
"Cache Attacks and Countermeasures: The Case of AES."
RSA Conference 2006 (CT-RSA), LNCS 3860, pp. 1-20.
DOI: 10.1007/11605805_1
Relevance: M1 (L1D miss patterns as attack signal)

[P10] Bernstein, D. J. (2005).
"Cache-timing attacks on AES." Technical report.
URL: <https://cr.yp.to/antiforgery/cachetiming-20050414.pdf>
Relevance: M1 (baseline for RDTSC-rate anomaly thresholds)

[P11] Percival, C. (2005).
"Cache Missing for Fun and Profit." BSDCan 2005.
URL: <https://www.daemonology.net/papers/htt.pdf>
Relevance: M4 (SMT/HT sibling core cross-contamination model)

Timing Attacks (Classical)

[P12] Kocher, P. C. (1996).
"Timing Attacks on Implementations of Diffie-Hellman, RSA, DSS, and Other Systems."
CRYPTO 1996, LNCS 1109, pp. 104-113.
DOI: 10.1007/3-540-68697-5_9
Relevance: M3 (constant-time comparison requirement for SHA-256 hash checks)

Survey / Countermeasure Overview

[P13] Ge, Q.; Yarom, Y.; Cock, D.; Heiser, G. (2018).
"A Survey of Microarchitectural Timing Attacks and Countermeasures
on Contemporary Hardware."
Journal of Cryptographic Engineering, 8(1), pp. 1-27.
DOI: 10.1007/s13389-016-0141-6
Relevance: Breadth reference; maps attack families to mitigations for threat model

[P14] Lipp, M. et al. (2020).
"Take A Way: Exploring the Security Implications of AMD's Cache Way Predictors."
ACM ASIACCS 2020, pp. 813-825.
DOI: 10.1145/3320269.3384746
Relevance: M1 (AMD-specific PMU event selection differences from Intel)

9.3 Standards, Specifications, and Technical Reports

CPU Architecture

[S1] Intel Corporation.
"Intel 64 and IA-32 Architectures Software Developer's Manual"
Combined Volumes 1-4. (Continuously revised.)
Key volumes: Vol. 3A – System Programming; Vol. 4 – MSR Reference.
URL: <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>
Internal alias: Intel SDM

[S2] AMD.
"AMD64 Architecture Programmer's Manual" Volumes 1-5. (Continuously revised.)
Vol. 2: System Programming; Vol. 3: General-Purpose and System Instructions.
URL: <https://www.amd.com/en/support/tech-docs>
Internal alias: AMD APM

Trusted Platform Module

[S3] Trusted Computing Group.
"TPM Library Specification, Family 2.0" Parts 1-4. Revision 01.59, 2019.
URL: <https://trustedcomputinggroup.org/resource/tpm-library-specification/>

NIST Special Publications

[S4] Chen, L. et al. (NIST). SP 800-90A Rev.1:
"Recommendation for Random Number Generation Using Deterministic Random Bit Generators."
NIST, 2015. DOI: 10.6028/NIST.SP.800-90Ar1

[S5] Cooper, D. A. et al. (NIST). SP 800-147:
"BIOS Protection Guidelines."
NIST, 2011. DOI: 10.6028/NIST.SP.800-147

[S6] Polk, T.; Regenscheid, A. (NIST). SP 800-155 (Draft):
"BIOS Integrity Measurement Guidelines."
NIST, 2011. URL: <https://csrc.nist.gov/publications/detail/sp/800-155/draft>

[S7] Barker, E. (NIST). SP 800-57 Part 1 Rev.5:
"Recommendation for Key Management."
NIST, 2020. DOI: 10.6028/NIST.SP.800-57pt1r5

Vendor Security Advisories

[S8] Intel Product Security Center. SA-00075 (INTEL-SA-00075):
"Intel Active Management Technology Escalation of Privilege."
Intel, 2017.

[S9] Intel Product Security Center. SA-00088 (INTEL-SA-00088):
 "Speculative Execution and Indirect Branch Prediction Side Channel Analysis Method."
 Intel, 2018.
 Relevance: M4 (IBRS/STIBP/IBPB selection via IA32_SPEC_CTRL)

[S10] Intel Product Security Center. SA-00233 (INTEL-SA-00233):
 "Microarchitectural Data Sampling Advisory."
 Intel, 2019.
 Relevance: M4 (VERW mitigation, MDS_NO flag, IA32_ARCH_CAPABILITIES bit 5)

[S11] Intel Product Security Center. SA-00161 (INTEL-SA-00161):
 "L1 Terminal Fault / Foreshadow Advisory."
 Intel, 2018.
 Relevance: M4 (L1TF_NO flag, IA32_FLUSH_CMD MSR 0x10B)

[S12] Microsoft Security Response Center (MSRC).
 "ADV180002: Guidance to mitigate speculative execution side-channel vulnerabilities."
 Microsoft, 2018. (Updated continuously.)
 Relevance: M3, M4 (Windows-level mitigation interactions)

Bus and I/O Specifications

[S13] PCI-SIG.
 "PCI Express Base Specification Revision 5.0."
 PCI-SIG, 2019.
 Relevance: M2 (ECAM base address formula, configuration space layout)

[S14] PCI-SIG.
 "PCI Bus Power Management Interface Specification Revision 1.2."
 PCI-SIG, 2004.
 Relevance: M2 (device capability registers)

9.4 Courses and Lecture Series

[L1] MIT OpenCourseWare – 6.858 Computer Systems Security
 Instructors: Frans Kaashoek, Nickolai Zeldovich (MIT CSAIL)
 URL: <https://ocw.mit.edu/courses/6-858-computer-systems-security-fall-2014/>
 Relevance: Threat modelling methodology, kernel security, privilege separation

[L2] Stanford CS 155 – Computer and Network Security
 Instructors: Dan Boneh, John Mitchell (Stanford University)
 URL: <https://cs155.stanford.edu/>
 Relevance: Side-channel fundamentals, memory safety, hardware security

[L3] CMU 15-410 / 15-605 – Operating System Design and Implementation
Instructors: David Eckhardt, Dave O'Hallaron (CMU)
URL: <https://www.cs.cmu.edu/~410/>
Relevance: Kernel scheduler, IRQ/Interrupt analogue, synchronisation primitives

[L4] USENIX Security Training – "Microarchitectural Side Channels"
Presenters: Yuval Yarom et al., USENIX Security 2019/2020
URL: <https://www.usenix.org/conference/usenixsecurity19>
Relevance: M1 attack primitives, PMU usage, flush-based defences

[L5] Intel Developer Zone – Performance Monitoring Unit (PMU) Tutorial Series
Authors: Intel Architecture Performance Monitoring Team
URL: <https://www.intel.com/content/www/us/en/developer/topic-technology/software-optimization-notice.html>
Relevance: M1 (IA32_PERFVTSEL encoding, PMI delivery, PEBS)

[L6] Microsoft Learn – Windows Driver Development Documentation
Authors: Microsoft WDK Team
URL: <https://learn.microsoft.com/windows-hardware/drivers/>
Relevance: All modules (WDM/KMDF conventions, IRQ rules, MDL handling)

[L7] Daira Hopwood – "Applied Cryptographic Hardware" (Real World Crypto, 2020)
URL: <https://rwc.iacr.org/2020/>
Relevance: TPM integration, hardware random number generation (M5)

10 Literature Map

NEW

The table below maps each driver module to its authoritative and secondary literature sources. Consult this table before making any implementation decision to identify the correct reference. Numbers in square brackets refer to textbooks in §9.1; letter codes to papers [P1–P14], standards [S1–S14], and courses [L1–L7] in §9.2–9.4.

Driver Module	Primary Sources	Secondary Sources
M1 — PMU / Cache Detection	S1 (SDM Vol.4), S2 (AMD APM Vol.3), P7, P8, P9, P10, L5	P13, P6
M2 — PCIe / IOMMU	S13, S14, S1 (SDM Vol.3A §29)	S12
M3 — Kernel Integrity	P1, P2, S12 (MSRC ADV180002), S5, S6	[1], [4], [5]
M4 — Cache Mitigation	S10, S11, P4, P5, P3, S1 (SDM Vol.4 §2.1), P11	P6, P13
M5 — Secure Comms / TPM	S3, S7, S4, [13], L7	[11], [12]
Threat model	P1–P6, P13, L1, L2	P7–P12
Patent prior art	P1–P14, S1–S14	[1]–[18]

Mark sources as **SUPERSEDED** in `literature/INDEX.md` when a newer revision is published. Never delete superseded sources — they are part of the patent prosecution history.

A Appendix A — Threat Model Maintenance

NEW

Literature is the primary input to the threat model. Every new paper or advisory should be evaluated against the current threat model using the following three-question framework.

A.1 Evaluation Framework

1. Does this attack fall within the current threat model?

Answer	Required Action
Yes, and the driver mitigates it	Update test coverage to include the specific attack variant
Yes, but the driver does not mitigate it	Open a gap item; assign to the owning module
No	Document the scoping decision explicitly in literature/INDEX.md

2. Does this attack require revising assumptions?

Example: A new attack that works across physical cores invalidates any assumption that physical core isolation is a complete defense. In that case, the SMT isolation logic in Module 4 must be re-evaluated.

3. Does this attack suggest new detection signals?

Example: a new timing pattern measurable by the PMU → add a PMU event counter definition to `pmu_detection.c` and update the threshold table in §3.2.

A.2 Source Versioning and Staleness

Trigger	Required Action
New Intel/AMD microarchitecture released	Re-read SDM Vol.4, check new CPUID leaves and MSRs
New CVE in the Spectre/MDS/LVI family	Read the advisory and research paper before the 90-day embargo lifts
WDK version update	Re-read affected API documentation for behavioral changes
Patent application filed by a competitor	Read the published application within 30 days
Pre-existing source superseded by a new revision	Update all extraction notes that cite the old revision

A.3 Reading Workflow Summary

1. Classify the document (type taxonomy: arch spec, paper, advisory...)
2. Apply the correct read order for that type
3. Produce an extraction note in literature/ using the template:
 - ## [Document Title]
 - Source: [Full citation with revision/date]
 - Read date: YYYY-MM-DD
 - Relevance: [Which module(s)]
 - ### Key Facts – specific, citable facts
 - ### Open Questions
 - ### Action Items
 - ### Conflicts with Other Sources
4. Link the extraction to any affected source files (code comments)
5. Update literature/INDEX.md
6. Evaluate against threat model (this appendix)
7. Run prior art analysis if novel (Appendix B)
8. Schedule a re-read trigger if source has a TTL

B **Appendix B — Prior Art Analysis**

NEW

A patent claim is only as strong as its prior art search. This appendix defines the four-step procedure for constructing a prior art map before drafting any novel claim.

B.1 Step 1 — Keyword Search

Search USPTO Full-Text, Google Patents, and Espacenet using:

- Technical terms: "microarchitectural buffer flush", "cache allocation technology", "performance monitoring unit anomaly detection"
- Problem-space terms: "side channel attack", "hardware keylogger detection", "kernel integrity verification"
- Inventor-space terms: search key researchers in the field (Kocher, Yarom, Van Bulck, etc.)

B.2 Step 2 — CPC Classification Search

CPC Code	Scope
G06F 21/57	Certifying or maintaining trusted computer platforms
G06F 21/55	Detecting local intrusion or implementing counter-measures
G06F 21/75	Protecting specific internal or peripheral components
H04L 9/06	Cryptographic mechanisms using block ciphers

B.3 Step 3 — Claim Chart Construction

For each novel technique, map it to the closest prior art. Example claim chart for the adaptive mitigation strategy:

Claim: Adaptive mitigation strategy selected from CPU capability probing			
Claim Element	Implementation	Closest Prior Art	Gap
----- ----- ----- -----			
Runtime CPU capability	ProbeCpuCapabilities() reading	Intel SA-00233 (software guide,	
Driver-level adaptive selection			
probing	IA32_ARCH_CAPABILITIES (MSR 0x10A)	not a driver claim)	is not
claimed in SA-00233			
Cheapest safe strategy	MITIGATION_STRATEGY enum selection	—	Novel
combination at driver level			
Per-CPU strategy state	g_CpuMit[MAXIMUM_PROCESSORS]	—	Novel
per-core granularity in Ring 0			

B.4 Step 4 — Claim Status Classification

Status	Meaning	Action
Novel	No prior art found for this element	Potential independent claim element
Anticipated	Prior art fully discloses this element	Do not claim; cite the prior art
Obvious	Combination of prior art renders it obvious	Weak claim; add evidence of non-obviousness
Design-around	Existing patent covers this approach	Implement differently to avoid infringement

B.5 Identifying the Inventive Step

The inventive step is not the individual components — it is the *combination* and the specific problem it solves. Document:

- What problem the prior art fails to solve
- How your specific combination solves it
- Why the combination is non-obvious (why someone skilled in the art would not arrive at it without your contribution)

Cross-reference this analysis with the module-to-literature map in §10 to ensure all relevant prior art is accounted for before filing.